

Rapport de test de charge — Application MicroCred

Performance

Date d'exécution : 24/07/2026 **Outil :** k6 v0.54+ (conteneur Docker `grafana/k6`) **Durée du test :** 7 minutes (7m00s) **Cible :** Backend FastAPI (`rollback-backup-security-manual-backend-1`), via `host.docker.internal:8000` **Charge simulée :** 6 rôles (superadmin, admin, support, chef_agence, portefeuille_manager, membre_comite), montée progressive jusqu'à 100 utilisateurs virtuels simultanés

1. Résumé exécutif

Indicateur	Valeur	Statut
Requêtes totales	11 550	—
Taux de succès global	77.35 %	⚠ Insuffisant
Taux d'échec global	22.64 %	🔴 Critique
Débit moyen	27.5 req/s	⚠ Faible pour la charge visée
Itérations interrompues	62	🔴 Timeouts
Seuils k6 définis (<code>errors<2%</code>), (<code>http_req_failed<2%</code>), (<code>p95<1000ms</code>)	Dépassés	🔴 Échec du test

Verdict global : l'application ne tient pas la charge de 100 utilisateurs simultanés dans son état actuel. Les résultats révèlent **deux catégories de problèmes distinctes** qu'il faut traiter séparément : des pannes fonctionnelles systématiques (indépendantes de la charge) et une vraie dégradation de performance sous charge, concentrée sur l'authentification.

2. Détail des anomalies observées

2.1 Endpoints en échec systématique (0 % de réussite)

Ces endpoints échouent à **100 % quel que soit le niveau de charge**, y compris avec très peu d'utilisateurs simultanés. Ce ne sont **pas des problèmes de performance** — ce sont des pannes fonctionnelles ou des restrictions d'accès à investiguer indépendamment du test de charge.

Rôle	Endpoint	Échecs / Total
support	GET /metrics/summary	368 / 368
support	GET /metrics/daily	366 / 366
support	GET /metrics/current-credits	366 / 366
support	GET /complaints	365 / 365
portefeuille_manager	GET /bonus/results	318 / 318
membre_comite	GET /metrics/snapshots	130 / 130
superadmin	GET /bonus/rules	92 / 92
admin	GET /users	147 / 147
admin	GET /imports/batches	145 / 145
admin	GET /bonus/rules	144 / 144

Causes possibles, à vérifier une par une (test manuel, un seul appel à la fois) :

1. **Droit d'accès manquant pour le rôle** (403 attendu) — dans ce cas, le endpoint est correctement protégé et le "problème" vient du script de test qui simule un usage qui n'a pas de sens pour ce rôle.
2. **Bug applicatif réel** (500 ou erreur non gérée) — l'endpoint plante indépendamment de la charge.
3. **Erreur dans le script de test** (mauvais chemin, mauvaise méthode HTTP, paramètre requis manquant).

Il est frappant que ces échecs touchent presque exclusivement des endpoints **d'administration ou de reporting spécifique** (`bonus`, `imports/batches`, `users`, `complaints`, `snapshots`) — cohérent avec l'hypothèse n°1 (droits par rôle), mais à confirmer avant toute action corrective.

2.2 Dégradation liée à la charge — Authentification

C'est le vrai signal de performance du test, à distinguer des pannes ci-dessus.

Métrique	Valeur
<code>login_duration</code> moyenne	4.06 s
<code>login_duration</code> médiane	286 ms
<code>login_duration</code> p90	803 ms
<code>login_duration</code> p95	59 987 ms (~60 s)

L'écart énorme entre la médiane (286 ms, correct) et le p95 (60 s, correspondant au timeout par défaut de k6) indique qu'**une partie significative des tentatives de connexion ne tiennent pas simplement lentement — elles ne répondent jamais** sous charge élevée. C'est cohérent avec :

- Un débit global plafonné à **27.5 requêtes/seconde**, alors que la charge visée (jusqu'à 100 VUs) en demanderait bien plus.
- **62 itérations interrompues**, très probablement bloquées en attente d'une réponse de login.

2.3 Comportement correct observé (bon signal)

Une fois connectés, les endpoints de lecture "normaux" (`/auth/me`, `/metrics/summary`) pour les rôles où c'est autorisé, (`/metrics/daily`, `/metrics/charts`) affichent des **taux de réussite de 97-99 %**, avec des temps de réponse HTTP globalement raisonnables (médiane 111 ms, p90 543 ms). Le problème n'est donc pas généralisé à toute l'application — il est concentré sur le **goulot d'étranglement du login**.

3. Diagnostic — causes probables du goulot d'étranglement login

Sans avoir encore inspecté la configuration du backend, voici les causes les plus courantes pour ce type de symptôme (login qui s'effondre en premier sous charge, alors que les lectures tiennent mieux), par ordre de probabilité :

#	Cause probable	Comment vérifier
1	Coût de hachage du mot de passe trop élevé (bcrypt/argon2 avec un facteur de coût élevé) — volontairement lent par sécurité, mais sature le CPU sous forte concurrence	Chercher <code>bcrypt</code> , <code>CryptContext</code> , <code>rounds=</code> ou <code>Argon2</code> dans le code d'authentification
2	Un seul worker Uvicorn — tout le traitement passe par un seul processus/cœur	Vérifier la commande de lancement (<code>uvicorn app.main:app</code>) sans <code>--workers</code>) ou le <code>Dockerfile</code> / <code>docker-compose.yml</code>
3	Pool de connexions SQLAlchemy trop petit (défaut souvent <code>pool_size=5</code>) — les requêtes de login font la queue pour une connexion DB	Vérifier <code>create_engine(..., pool_size=..., max_overflow=...)</code> dans <code>app/db/session.py</code>
4	Requête de login non optimisée (recherche sans index sur l'email, jointures superflues)	<code>EXPLAIN ANALYZE</code> sur la requête SQL du login
5	Rate limiting / anti-bruteforce trop agressif — votre rapport de sécurité confirme un blocage après tentatives invalides répétées ; sous forte charge de test, ce mécanisme peut se déclencher et bloquer des comptes de test légitimes	Vérifier les logs applicatifs pendant le test pour repérer des blocages de compte

4. Recommandations priorisées

Priorité 1 — Bloquant, à traiter avant tout nouveau test de charge

1. **Investiguer les 10 endpoints à 0 % de réussite** (section 2.1) un par un, manuellement, par rôle. Distinguer clairement "accès refusé attendu" (rien à corriger, ajuster le script de test) de "bug réel" (à corriger dans le code).
2. **Identifier la cause du goulot d'étranglement login** (section 3) en vérifiant dans l'ordre : nombre de workers Uvicorn → taille du pool SQLAlchemy → coût du hachage de mot de passe.
3. **Vérifier les logs backend pendant un test de charge** pour voir si le mécanisme anti-bruteforce (mentionné dans le rapport de sécurité) se déclenche à tort sur les comptes de test.

Priorité 2 — Configuration à ajuster une fois la cause confirmée

Levier	Action suggérée	Justification
Workers Uvicorn	Lancer avec <code>--workers N</code> ($N \approx$ nombre de cœurs CPU disponibles), ou passer par Gunicorn en superviseur de workers Uvicorn	Permet de traiter plusieurs requêtes en parallèle sur plusieurs cœurs, au lieu d'un seul processus séquentiel
Pool SQLAlchemy	Augmenter <code>pool_size</code> et <code>max_overflow</code> dans <code>create_engine()</code> , en cohérence avec <code>max_connections</code> de PostgreSQL	Évite que les requêtes DB fassent la queue sous forte concurrence
PostgreSQL	Vérifier/augmenter <code>max_connections</code> côté serveur Postgres pour supporter le nouveau pool	Le pool applicatif ne sert à rien si la base elle-même plafonne les connexions
Hachage mot de passe	Mesurer le temps CPU d'un hachage seul (hors réseau/DB) ; si $> 200\text{-}300$ ms, envisager un <code>cost_factor</code> légèrement réduit en le compensant par d'autres mesures de sécurité (rate limiting, MFA) — à valider avec l'équipe sécurité, ne pas baisser ce paramètre sans arbitrage	Sécurité vs performance — décision qui doit rester consciente, pas un simple réglage technique
Rate limiting	Si confirmé comme cause de faux positifs en test, prévoir une liste d'exclusion (allowlist IP ou compte) pour les environnements de test de charge, sans l'assouplir en production	Ne pas affaiblir une protection de sécurité pour "faire passer" un test

Priorité 3 — Méthodologie de test, pour la suite

1. **Retester par paliers progressifs** plutôt que de sauter directement à 100 VUs : refaire le test avec `PEAK_VUS=20`, puis `50`, pour identifier précisément le seuil où la dégradation commence, plutôt que de tout observer mélangé à 100.
2. **Isoler le login du reste** : lancer un scénario k6 dédié qui ne teste QUE le login en charge, pour confirmer que c'est bien le goulot d'étranglement principal avant d'optimiser autre chose.
3. **Répéter le test après chaque correctif**, un changement à la fois (d'abord les workers, puis le pool DB, etc.), pour savoir lequel a réellement eu de l'effet plutôt que de tout changer en même temps.
4. **Ajouter un test de charge dans la routine avant chaque mise en production majeure**, une fois la configuration stabilisée, pour détecter les régressions de performance tôt.

5. Sur l'objectif "rendre l'application parfaite"

Un point de cadrage honnête : aucune application n'est "parfaite" au sens absolu — il y a toujours un compromis entre performance, coût d'infrastructure, sécurité et complexité de maintenance. Ce qui est atteignable et mesurable, c'est de définir un **objectif de capacité explicite** (par exemple : "l'application doit supporter 100 utilisateurs simultanés avec un p95 sous 1 seconde et moins de 1 % d'erreurs") et de itérer jusqu'à l'atteindre, en revalidant à chaque changement.

Les résultats actuels donnent une base de référence claire (77 % de succès, p95 login à 60s) pour mesurer les progrès des prochaines itérations.

6. Prochaines étapes suggérées

1. Corriger/expliciter les 10 endpoints à 0 %.
2. Vérifier la configuration Uvicorn/SQLAlchemy/Postgres actuelle (nombre de workers, pool_size, max_connections).
3. Relancer un test de charge ciblé uniquement sur le login, à faible puis moyenne charge.
4. Une fois le login stabilisé, relancer le test multi-rôles complet pour confirmer l'amélioration globale.